

MANIPAL SCHOOL OF INFORMATION SCIENCES

CRYPTOLOGY LAB — ASSIGNMENT 3

[Introduction to Python for Cryptology](#)

Student Name: Ribhav Devprasad

Registration No: 261100690002

Course: Cryptology Lab

Part A — Python Basics and Debugging

Q1. Debug Programs and Identify Error Types

```
# Program 1 – Error Type: SyntaxError (Unterminated string literal)
print("Welcome to Cryptology Lab)

# Program 2 – Error Type: NameError (Misspelled variable name)
message = "Python Programming"
print(mesage)

# Program 3 – Error Type: TypeError (Unsupported operand types for +: 'int' and 'str')
num = input("Enter a number: ")
result = num + 10
print(result)
```

Q2. Display Formatted Strings

```
print("Welcome to Cryptology Lab")
print("It's interesting to learn Python.")
print('Students, "Welcome to the Cryptology Lab".')
```

Q3. Arithmetic Operations Pipeline

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print(f"Addition: {num1} + {num2} = {num1 + num2}")
print(f"Subtraction: {num1} - {num2} = {num1 - num2}")
print(f"Multiplication: {num1} * {num2} = {num1 * num2}")

if num2 != 0:
    print(f"Division: {num1} / {num2} = {num1 / num2}")
    print(f"Integer division: {num1} // {num2} = {num1 // num2}")
    print(f"Remainder: {num1} % {num2} = {num1 % num2}")
else:
    print("Division, Integer division, and Remainder by zero are undefined.")

print(f"Exponentiation: {num1} ** {num2} = {num1 ** num2}")
```

Q4. Floating-Point Number Evaluation

```
val = float(input("Enter a floating-point number: "))

print("Data type:", type(val))
print("Absolute value:", abs(val))
print("Rounded to two decimal places:", round(val, 2))
```

Part B — Data Types and Collections

Q5. Python Data Types Demonstration

```
v_int = 15
v_float = 10.5
v_complex = 3 + 4j
v_str = "Cryptology"
v_list = [1, 2, 3]
v_tuple = (10, 20)
v_set = {1, 2, 3}
v_dict = {"cipher": "RSA", "bits": 2048}

vars_list = [v_int, v_float, v_complex, v_str, v_list, v_tuple, v_set, v_dict]

for item in vars_list:
    print(f"Value: {item} | Type: {type(item)}")
```

Q6. String Immutability vs List Mutability

```
# String Immutability Experiment
try:
    test_str = "Python"
    test_str[0] = "J"
except TypeError as e:
    print(f"String Error Captured: {e}")

# List Mutability Experiment
test_list = [10, 20, 30]
print("Original List:", test_list)
test_list[0] = 99
print("Modified List:", test_list)
```

Q7. Prime List Operations

```
primes = [2, 3, 5, 7, 11]

print("First three elements:", primes[:3])
print("Last element:", primes[-1])

primes.append(13)
print("After append:", primes)

primes.extend([17, 19])
print("After extend:", primes)

print("Total number of elements:", len(primes))
```

Part C — String Operations

Q8. String Slicing and Reversal

```
message = "CRYPTOLOGY USING PYTHON"

print("Length of string:", len(message))
print("First five characters:", message[:5])
print("Last six characters:", message[-6:])
print("Characters from positions 3 to 8:", message[3:9])
print("Reverse order:", message[::-1])
```

Q9. String Normalization & Replacement

```
message = "CRYPTOLOGY USING PYTHON"

print("Lowercase:", message.lower())
print("Back to uppercase:", message.lower().upper())
print("Does 'PYTHON' exist?:", "PYTHON" in message)
print("Replaced string:", message.replace("PYTHON", "PROGRAMMING"))
```

Q10. Whitespace Stripping

```
message = "CRYPTOLOGY USING PYTHON"
no_spaces = message.replace(" ", "")
print("Without spaces:", no_spaces)
```

Q11. ASCII/Unicode Evaluation Loop

```
message = "CRYPTOLOGY USING PYTHON"
for char in message:
    print(f"{char} -> {ord(char)}")
```

Q12. ASCII Array Decoding

```
ascii_values = [65, 67, 73, 80, 72, 69, 82]
decoded_word = "".join([chr(val) for val in ascii_values])
print("Identified Word:", decoded_word)
```

Part D — Text Processing for Cryptology

Q13. Plaintext Preprocessing Pipeline

```
user_input = input("Enter a message: ")
processed = user_input.strip().upper().replace(" ", "")
print("Output:", processed)
```

Q14. Character Grouping into Blocks of 5

```
text = input("Enter a string: ").replace(" ", "")
for i in range(0, len(text), 5):
    print(text[i:i+5])
```

Q15. Block Padding with 'X'

```
text = input("Enter a string: ").replace(" ", "")
remainder = len(text) % 5

if remainder != 0:
    text += "X" * (5 - remainder)

for i in range(0, len(text), 5):
    print(text[i:i+5])
```

Q16. Character Frequency Analysis

```
text = "CRYPTOLOGYISINTERESTING"
freq = {}

for char in text:
    freq[char] = freq.get(char, 0) + 1

print("Character Frequencies:", freq)
most_frequent = max(freq, key=freq.get)
print(f"Most frequent character: '{most_frequent}' value: {freq[most_frequent]}")
```

Q17. Percentage Frequency Distribution

```
text = "CRYPTOLOGYISINTERESTING"
total_chars = len(text)
freq = {}

for char in text:
    freq[char] = freq.get(char, 0) + 1

for char, count in freq.items():
    percentage = (count / total_chars) * 100
    print(f"{char}: {percentage:.2f}%")
```

Part E — Mathematical Foundations for Cryptology

Q18. Modular Arithmetic Computations

```
print("29 mod 26 =", 29 % 26)
print("55 mod 26 =", 55 % 26)
print("78 mod 26 =", 78 % 26)
print("-3 mod 26 =", -3 % 26)
print("-29 mod 26 =", -29 % 26)

val = int(input("\nEnter an integer: "))
print(f"{val} mod 26 = {val % 26}")
```

Q19. Alphabet Representation (A=0 ... Z=25)

```
# Character to number
char_in = input("Enter character: ").upper()
num_out = ord(char_in) - ord('A')
print(f"{char_in} -> {num_out}")

# Number to character
num_in = int(input("Enter number (0-25): "))
char_out = chr(num_in + ord('A'))
print(f"{num_in} -> {char_out}")
```

Q20. Prime Number Verification & Range Generator

```
def check_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

num = int(input("Enter number to check: "))
print(f"Is {num} prime?: {check_prime(num)}")

start = int(input("Enter lower range: "))
end = int(input("Enter upper range: "))
primes_found = [x for x in range(start, end + 1) if check_prime(x)]
print("Primes in range:", primes_found)
```

Q21. GCD and Coprimality Evaluation

```
import math

def coprime_eval(a, b):
    gcd_val = math.gcd(a, b)
    is_coprime = (gcd_val == 1)
    print(f"Pair ({a}, {b}) -> GCD: {gcd_val} | Coprime: {is_coprime}")

coprime_eval(15, 26)
coprime_eval(18, 24)
coprime_eval(17, 26)
```

Q22. Modular Multiplicative Inverse (mod 26)

```
def ext_gcd_mod_inverse(b, m=26):
    m0, x0, x1 = m, 0, 1
    if m == 1:
        return None
    while b > 1:
        if m == 0:
            return None
        q = b // m
        b, m = m, b % m
        x0, x1 = x1 - q * x0, x0
    if x1 < 0:
        x1 += m0
    return x1

b_val = int(input("Enter integer b: "))
inverse = ext_gcd_mod_inverse(b_val, 26)
if inverse:
    print(f"Modular Multiplicative Inverse of {b_val} mod 26 is: {inverse}")
else:
    print(f"Inverse does not exist for {b_val} mod 26.")
```

Part F — Challenge / Bonus

Q23. Client-Server Socket Communication

```
import socket

# Server
server = socket.socket()
server.bind(('localhost', 5000))
server.listen(1)

# Client sends message
client = socket.socket()
client.connect(('localhost', 5000))

conn, addr = server.accept()
client.send(b"Hello Server")

# Server receives message
msg = conn.recv(1024).decode()
print("Received:", msg)

client.close()
server.close()
```